

Homotopy Type Theory: Chapter 1

Alex L.

February 21, 2026

1 Type Theory

1.1 Type Theory vs Set Theory

Definition: (Rules and Judgements)

In a deductive system, we have a collection of **rules**, and by applying those rules, we can create things called **judgements**.

Definition: (Type)

In type theory, the most basic judgement is:

$$a : A$$

and means " a has type A " or "the type A has a proof a ", or in homotopy type theory, " a is a point in the space A ". In type theory, every element has a type.

Definition: (Equality)

In type theory, there are two types of equality. Given $a : A$, $b : A$, we can create a new type

$$a =_A b$$

which may or may not be empty (but we say a type is "inhabited"). This is called **propositional equality**.

We also have judgemental equality, saying

$$a \equiv b$$

on the same level as a statement like $a : A$. This is basically saying, "by definition, a is equal to b ", and is called **judgemental equality**.

Example:

If we let a function $f : \mathbb{N} \rightarrow \mathbb{N}$ be defined as $f(x) \equiv x^2$, then judgementally, (by definition), $f(3) \equiv 3^2$.

Remark:

Judgemental equality can't be used as an assumption or negation. For instance, it doesn't make sense to say "if x is definitionally equal to y , then z is not definitionally equal to w ". You're the one deciding lil bro.

The two forms of judgement, typing and equality, form the basis of type theory.

Definition: (Context)

We also may have implicit assumptions about type. For example, typing

$$m + n : \mathbb{N}$$

as above gives the implicit assumption that $m : \mathbb{N}$ and $n : \mathbb{N}$. These implicit assumptions are called **context**.

1.2 Function Types

Definition: (Function Types)

Given types A, B , we can construct the type

$$A \rightarrow B$$

of **functions** with domain A and codomain B . Functions are primitive, in the sense that we create them and tell you what you can do with them.

Given some function $f : A \rightarrow B$, you can **apply** the function f to some $a : A$ to get $f(a) : B$, and we call this the **value** at a , and also note that we do notate $f(a)$ as fa often.

In order to define a function $f : A \rightarrow B$, we often say $f(x) = \Phi$ where Φ is some expression that may or may not depend on x , and we want to check that $\Phi : B$ assuming $x : A$.

Definition: (λ -abstraction)

We can also define functions anonymously via λ -abstraction as follows:

$$(\lambda(x : A).\Phi) : A \rightarrow B$$

after the λ , we have parameters to take in, in this case just some x of type A , and the dot signifies it returns Φ with x substituted for whatever value of x was provided.

Example:

For some $y : B$, we have $(\lambda(x : A).y) : A \rightarrow B$, the **constant function**.

Example:

We may also use implicit λ -abstraction, meaning we can write

$$g(x, -)$$

to denote

$$\lambda y. g(x, y)$$

Definition: (β -conversion)

By definition,

$$(\lambda x. \Phi)(a) \equiv \Phi'$$

where Φ' is Φ but all occurrences of x have been replaced by a . This is called the **computation rule**, or in λ calculus, the β -conversion

Example:

$$(\lambda x. x + x)(2) = 2 + 2$$

(not necessarily equal to 4 we haven't proved that yet)

Definition: (η -expansion)

We also define:

$$\lambda x. f(x) \equiv f$$

and this is called the **uniqueness principle for function types**, because this is saying that f is uniquely determined by its values for all x in a type.

It's also called the η -expansion in λ calculus

Remark:

Consider

$$(f(x) \equiv \lambda y : \mathbb{N}. x + y) : \mathbb{N} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$$

This function takes in some x , and returns another function of a variable y .

What happens if we do $f(y)$. If we just replace all occurrences of x with y as defined by β -reduction, we get that $f(y) = \lambda y. y + y$. But there is an issue. Previously, the y was referring to what we β -reduced by, or what we put into $f(y)$. But now, the y is referring to the argument of the anonymous function, λy . These are different y 's.

The solution is that $f(y) = \lambda z. y + z$, we basically rename the variable.

Definition: (Currying)

If we want a function in two variables, we define a function $f : A \rightarrow B \rightarrow C$ via:

$$f(x, y) \equiv \Phi$$

and

$$f \equiv \lambda x : A. \lambda y : B. \Phi$$

this is called **currying**, we basically return another function.

1.3 Universes

Definition: (Universes)

A **universe** is a type whose elements are also types.

We might wish that there be a universe which includes all types, but we actually end up with the type `False` being inhabited (which implies that the type `False` is actually true, which might not be a great outcome).

We instead have a hierarchy of universes:

$$\mathcal{U}_0 : \mathcal{U}_1 : \mathcal{U}_2 : \dots$$

such that each universe is an element of the next.

We also stipulate that the universes are cumulative, so if some type $A : \mathcal{U}_n$, it is also an element of $A : \mathcal{U}_{n+1}$ and so on...

Remark:

When we say A is a type, we say it is an element of some universe,

$$A : \mathcal{U}_i$$

we also usually omit the numberings (because we generally don't care to explicitly enumerate these things), so

$$\mathcal{U}_n : \mathcal{U}_{n+1}$$

becomes

$$\mathcal{U} : \mathcal{U}$$

This method of omitting indices is called **typical ambiguity**.

Definition: (Families of Types)

A type-valued function

$$B : A \rightarrow \mathcal{U}$$

is called a **family of types**

Example:

The family of finite sets can be expressed as

$$\text{Fin}(n) : \mathbb{N} \rightarrow \mathcal{U}$$

where $\text{Fin}(n)$ is a type with exactly n elements,

$$\text{Fin}(n) = \{0_n, 1_n, \dots, (n-1)_n\}$$

with the subscripts to make it clear that $0_4 : \text{Fin}(3)$ is not be the same element as $0_4 : \text{Fin}(4)$ and none of these are equal to $0 : \mathbb{N}$

Example:

The constant type family

$$B : \mathcal{U}$$

given by

$$\lambda x : A. B : A \rightarrow \mathcal{U}$$

Example:

There is no type family

$$\lambda(i : \mathbb{N}) : \mathcal{U}_i$$

because firstly, there is no universe large enough, and secondly, we don't even index universes with natural numbers?

2 Dependent Types (Π -types)

Definition: (Dependent Types)

Dependent types are a generalization of function types. Elements of a dependent type are functions whose output type isn't fixed, it can vary depending on the input. We notate dependent types by:

$$\Pi_{(x:A)} B(x)$$

They are called Π -types because they are pretty closely linked to cartesian products.

Via λ -abstraction:

$$f \equiv \lambda(x : A). \Phi : \Pi_{x:A} B(x)$$

And if we apply β -conversion, we get an output $f(a) : B(a)$, where $f(a)$ is an element of the type $B(a)$

Example:

Consider the function

$$\text{fmax} : \Pi_{(n:\mathbb{N})} \text{Fin}(n+1)$$

which returns

$$\text{fmax}(n) \equiv n_{n+1} : \text{Fin}(n+1)$$

Definition: (Polymorphic Functions)

A **polymorphic** function is a function which itself may vary based on the type of it's input. In a sense, it can accept inputs of multiple types, and it's behavior changes based on which type it's input belongs to.

Example:

The polymorphic identity function:

$$\text{id} : \Pi_{A:\mathcal{U}} A \rightarrow A$$

is given by

$$\text{id} \equiv \lambda(A : \mathcal{U}). \lambda(x : A). x$$

Example:

The swap function, which given a curried function, changes the order in which you apply the β -reductions:

$$\text{swap} : \Pi_{A:\mathcal{U}} \Pi_{B:\mathcal{U}} \Pi_{C:\mathcal{U}} (A \rightarrow B \rightarrow C) \rightarrow (B \rightarrow A \rightarrow C)$$

which takes

$$\text{swap} \equiv \lambda(A : \mathcal{U}). \lambda(B : \mathcal{U}). \lambda(C : \mathcal{U}). (\lambda(x : A). \lambda(y : B). \lambda(z : C). \Phi). (\lambda(y : B). \lambda(x : A). \lambda(z : C). \Phi)$$

2.1 Product Types

Remark:

When we introduce a new type, we generally specify these things:

1. **formation rules:** how to form new types of this kind
2. **construction rules:** how to construct new elements in types of this kind
3. **elimination rules:** how to use elements of this type (functions have one eliminator, that being applying an input to a function, aka β -reduction)
4. **computation rules:** how eliminators act on constructors (for functions, the computation rule is that $(\lambda x.\Phi)(a)$ is by definition, equal to substituting all occurrences of x in Φ for a)
5. **uniqueness principle:** an optional principle which characterizes maps in/out of the type (for functions, the uniqueness principle is that $f \equiv \lambda x.f(x)$). Sometimes, uniqueness isn't a judgement but instead it is propositional (can be derived from other judgements), in which case we call it a **propositional uniqueness principle**.

Definition: (Cartesian Product)

Given types $A, B : \mathcal{U}$, let $A \times B : \mathcal{U}$ be a type called their **cartesian product**. For $a : A$ and $b : B$, judgementally, $(a, b) : A \times B$

Elimination rule: Given some $g : A \rightarrow (B \rightarrow C)$, we define some function

$$\begin{aligned} f &: A \times B \rightarrow C \\ f((a, b)) &\equiv g(a)(b) \end{aligned}$$

Definition: (Unit Type)

The **unit type** is $\mathbf{1}$, and there is a single element of this type,

$$\star : \mathbf{1}$$

Example:

We can have projection functions

$$\begin{aligned} \text{pr}_1 &: A \times B \rightarrow A \\ \text{pr}_2 &: A \times B \rightarrow B \end{aligned}$$

such that

$$\text{pr}_1((a, b)) \equiv a$$

and

$$\text{pr}_2((a, b)) \equiv b$$

Definition: (Recursor)

The **recursor** is defined as

$$\text{rec}_{A \times B} : \Pi_{C:\mathcal{U}}(A \rightarrow B \rightarrow C) \rightarrow (A \times B \rightarrow C)$$

where

$$\text{rec}_{A \times B}(C, g, (a, b)) \equiv g(a)(b)$$

The recursor is actually defined on things called inductive types, which are a generalized version of product types, where they will actually be recursive.

Example:

$$\text{pr}_1 \equiv \text{rec}_{A \times B}(A, \lambda a.\lambda b.a)$$

and

$$\text{pr}_2 \equiv \text{rec}_{A \times B}(B, \lambda a.\lambda b.b)$$

Exercise:

Define the recursor in terms of the projection functions and vice versa.

Solution:

(Recursor in terms of Projection)

$$\text{rec}_{A \times B} \equiv \lambda(C : \mathcal{U}). \lambda(g : A \rightarrow B \rightarrow C). \lambda(x : A \times B). g(\text{pr}_1(x))(\text{pr}_2(x))$$

(Projections in terms of recursor)

$$\text{pr}_1 \equiv \lambda(C : \mathcal{U}). \lambda(x : A \times B). \text{rec}_{A \times B}(\lambda(a : A). \lambda(b : B). a)(x)$$

$$\text{pr}_2 \equiv \lambda(C : \mathcal{U}). \lambda(x : A \times B). \text{rec}_{A \times B}(\lambda(a : A). \lambda(b : B). b)(x)$$

Proposition:

There exists a function of type

$$\text{uniq}_{A \times B}((a, b)) : \prod_{x : A \times B} ((\text{pr}_1(x), \text{pr}_2(x)) =_{A \times B} x)$$

showing that every element of $A \times B$ is equal to a pair.

Proof:

Let $\text{uniq}_{A \times B}((a, b)) \equiv \text{refl}_{(a, b)}$, where $\text{refl}_x : x =_A x$ for any $x : A$.

This, combined with the identity $(\text{pr}_1((a, b)), \text{pr}_2((a, b))) \equiv (a, b)$ means that

$$\text{refl}_{(a, b)} : (a, b) =_{A \times B} (a, b)$$

is also of type

$$\text{refl}_{(a, b)} : (\text{pr}_1((a, b)), \text{pr}_2((a, b))) \equiv_{A \times B} (a, b)$$

and this is well typed since they are judgementally equal.